LARA: Latent Action Routing for Embodied Control

Discovering Action Experts from Demonstrations and Calibrating Sparse Routers by Control Utility

Haoran Zheng

Draft manuscript

Abstract

Vision-language-action (VLA) policies are commonly trained from goal-level language paired with robot trajectories. Yet a single instruction can induce a multi-modal short-horizon action distribution: a robot may approach, transport, align, insert, reorient, retry, or recover through different local behaviors. A dense action head must represent all of these modes through one shared computation path, while a naively routed mixture-of-experts (MoE) can collapse into generic load balancing or redundant experts that relearn the same full action predictor. This paper proposes *LARA*, a latent-action routing framework for VLA control. LARA does not predefine experts as semantic, contact, spatial, or recovery modules. Instead, a posterior latent-action encoder compresses demonstrated future action chunks into latent action codes; a shared latent-action policy predicts the common action chunk; residual experts specialize by improving that shared prediction; and a deployable router selects sparse residual experts from the current multimodal context alone. For deployment, the router can be implemented as a two-level mechanism: an episode-level pool router selects a resident expert subset, and a chunk-level router selects top-k experts inside that pool. A final counterfactual utility objective calibrates routing toward experts that improve downstream control, rather than merely reconstructing demonstrations. LARA uses ordinary goal-level language and robot trajectories, without dense step-level language annotations or high-frequency video generation. The intended empirical claim is deliberately narrow: under matched active inference compute and resident-expert budgets, latent-action residual experts and utility-calibrated sparse routing should improve closed-loop success, policy-ranking fidelity, route calibration, and compute-success Pareto efficiency relative to dense VLA and generic sparse MoE baselines.

1 Introduction

Robot manipulation is usually specified at the level of intention. A person says, "put the cap on the shaver" or "place the cup on the shelf," while the body produces a sequence of short motor behaviors without verbalizing every intermediate muscle contraction, contact transition, and corrective adjustment. This observation suggests a different way to build routed VLA systems. Rather than predefining experts as semantic, spatial, contact, articulation, or recovery modules, the model should discover behavior modes from trajectories and learn when each mode is useful for control.

A tempting design is to divide experts according to human-interpretable stages. That design is easy to explain, but it risks imposing an external taxonomy that the policy itself should not need. LARA instead takes the action chunk as the unit of discovery. Given the current multimodal context and a demonstrated future chunk, a posterior encoder infers a latent action code. Experts are then softly bound to regions of the learned behavior manifold by posterior responsibility. Their semantics are not fixed in advance; if an expert later appears approach-like, transport-like, or precision-like, that description is a post-hoc interpretation of the learned routing structure.

The central hypothesis is:

> Robot demonstrations contain latent short-horizon behavior modes. If these modes are discovered from action chunks, softly bound to MoE experts by posterior responsibility, and routed at deployment by a context-only router calibrated with downstream control utility, then a VLA policy can use conditional capacity more effectively under a fixed active compute budget.

This is not a claim that MoE removes dependence on data coverage or hardware constraints. Generalization remains bounded by the diversity and quality of robot data, and inference remains bounded by the deployable compute budget. The claim is instead about *control generalization efficiency*: for the same goal-level text, trajectory data, active inference compute, and resident-expert budget, can a latent-action MoE policy reduce behavioral mode interference and improve closed-loop control?

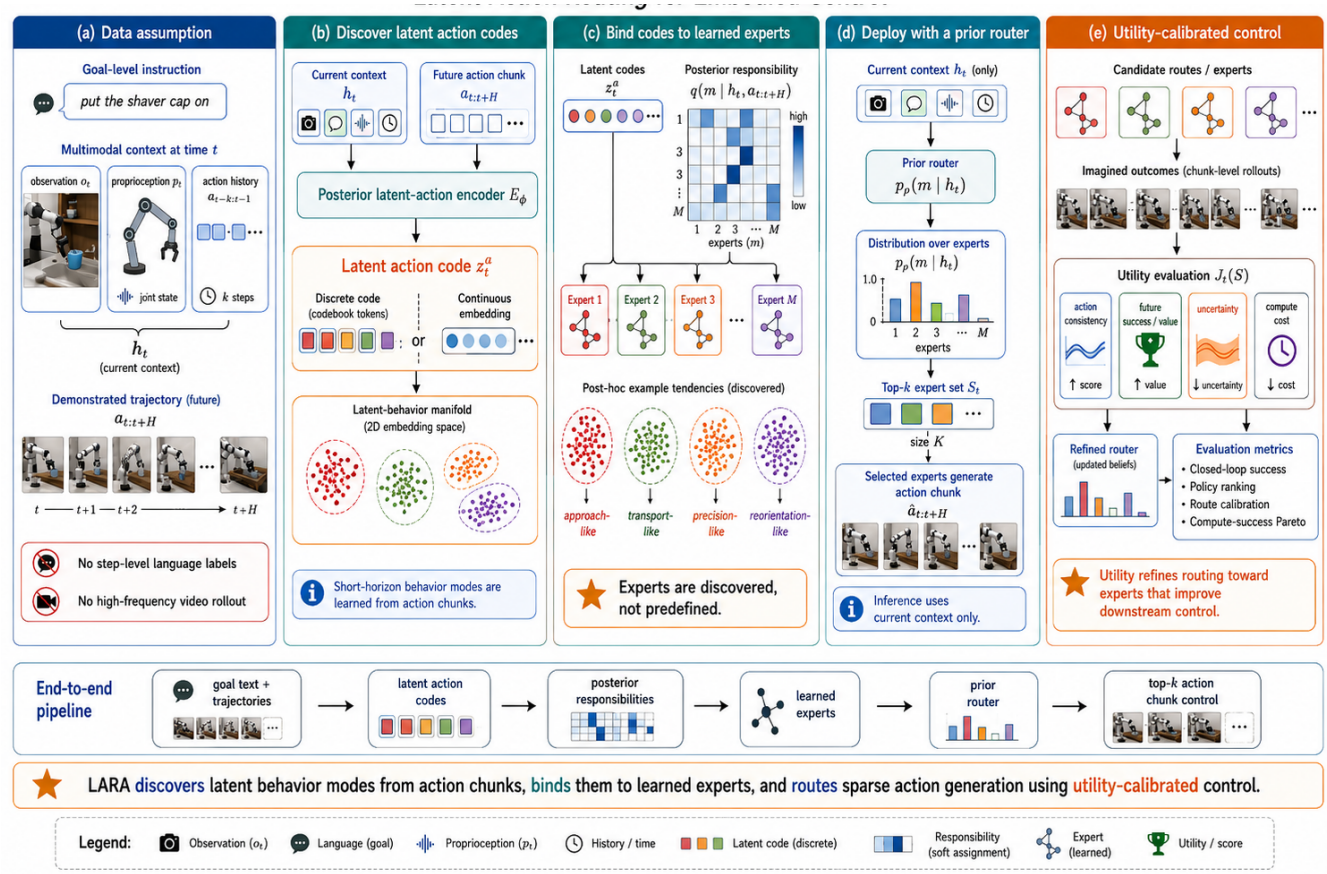

Figure 1: Overview of LARA. Given ordinary goal-level language and robot demonstrations, LARA first discovers latent action codes from demonstrated future action chunks, without dense step-level language labels. The codes organize demonstrations into a latent behavior manifold. A shared latent-action policy captures common control structure, while routed residual experts learn corrections that improve the shared action chunk for particular behavior regions. A deployable router then predicts sparse expert selection from the current multimodal context alone. In deployment-oriented settings, this router can be two-level: an episode-level pool router selects a resident expert subset, while a chunk-level router selects top-k experts inside the pool. Counterfactual utility fine-tuning calibrates routing toward experts that improve chunk-level control under matched active inference compute. Expert semantics are discovered post-hoc rather than manually predefined, and the method avoids high-frequency visual rollout generation.

Contributions.

1. We formulate VLA expert routing around *latent action codes* learned from demonstrated action chunks, not human-defined manipulation stages or dense step-level text annotations.
2. We propose an improvement-based posterior responsibility mechanism that softly binds demonstrations and latent action codes to learned residual experts over a shared latent-action policy.
3. We train a deployable router by distilling posterior expert responsibilities into a context-only route predictor, optionally factorized into an episode-level pool router and a chunk-level top-k router for resident-expert efficiency.
4. We refine routing with counterfactual control utility, so experts are selected for downstream control value rather than only demonstration reconstruction.
5. We define a matched-compute and matched-resident-expert experimental protocol that tests closed-loop success, policy ranking, route calibration, subset retention, and compute-success Pareto efficiency without high-frequency visual generation.

2 Related Work and Design Boundary

Action-chunk policies and VLA control. Modern robot policies increasingly predict short action chunks rather than individual motor commands, as in diffusion-style visuomotor policies and action-chunking transformer policies [3]. LARA follows this chunked-control view: a goal-level instruction conditions a sequence of short-horizon behaviors, and the policy should discover the latent action modes that explain those behaviors. LARA is also compatible with larger VLA backbones such as RT-style policies, OpenVLA, or Octo-like generalist policies [4, 5, 6, 7], but the core claim is not tied to a specific large VLA implementation.

Sparse MoE and expert specialization. Sparse MoE models activate only a subset of experts per input, enabling conditional computation and larger total capacity at a fixed active compute budget [1, 2]. However, generic MoE routing does not guarantee useful modularity. Experts may specialize according to low-level or unstable patterns, collapse to a few dominant modules, or fail to remain useful when subsets are deployed independently. Recent language-model MoE work on emergent modularity shows that subset usability should be designed and evaluated directly, rather than assumed from sparsity alone [10]. LARA incorporates the same engineering principle at the level of robot trajectories and action chunks: sparsity is useful only if the selected experts remain coherent and deployable under resident-memory budgets.

World models without dense video rollout. World models for control show the value of predicting future latent states for planning or policy improvement [8, 9]. LARA narrows this idea to chunk-level control. It does not require generating every intermediate visual frame. Instead, the optional transition target predicts a chunk-boundary latent state that is sufficient for the next decision step. This preserves control-relevant future supervision while avoiding high-frequency video generation.

3 Problem Setup

We assume a dataset of robot demonstrations

$$\mathcal{D} = \{\tau_i\}_{i=1}^{N}, \qquad \tau_i = (g_i, o^i_{1:T_i}, p^i_{1:T_i}, a^i_{1:T_i}), \tag{1}$$

where o_t is an observation such as RGB or multi-view RGB, p_t is proprioception, a_t is a robot action, and g is a goal-level language instruction. The key data assumption is intentionally weak: g is an ordinary task description such as "put the shaver cap on," not a dense list of step-by-step action labels. Intermediate behavioral structure is expected to be learned from trajectories rather than specified by language annotations.

Some benchmarks additionally provide sparse rewards, success indicators, or termination annotations. We denote such optional signals by

$$\mathcal{Y}_\tau = \{r_{1:T}, y_\tau, \delta_{1:T}\} \tag{2}$$

when available, where r_t is a reward, y_τ is an episode-level success label, and δ_t is an optional termination indicator. These signals are not required for latent-action discovery, expert binding, or prior-router distillation; they are used only for evaluation, optional value/progress heads, or utility calibration.

A multimodal encoder maps the current context to a control state

$$h_t = F_\theta(o_t, g, p_t, a_{t-k:t-1}). \tag{3}$$

The model predicts or reconstructs a short action chunk

$$a_{t:t+H-1} = (a_t, a_{t+1}, \ldots, a_{t+H-1}), \tag{4}$$

rather than only a single action. Chunked control matters because a high-level intention is rarely expressed by one motor command; it is expressed by a short behavior sequence. Near the end of an episode, chunks that exceed the trajectory boundary are either omitted or evaluated with a valid-step mask.

In deployment we distinguish the *prediction horizon* H_p from the *execution horizon* H_e. The policy may predict a long future chunk $\hat{a}_{t:t+H_p-1}$, while the controller executes only the first H_e actions before taking a new real observation and replanning. For the SO101 setting considered here, the default 30Hz configuration is

$$H_p = 60, \qquad H_e = 10. \tag{5}$$

Thus each policy call predicts two seconds of future actions, but only the first one third of a second is actually sent to the robot before the next closed-loop update.

We also allow a chunk-level state transition target. Instead of generating every intermediate image frame, the model predicts a control-sufficient future latent state at the chunk boundary,

$$\hat{h}_{t+H} = T_\psi(h_t, z_t^a), \tag{6}$$

where z_t^a is the latent action code introduced in the next section. The target state is obtained by encoding the real observation and proprioception at the chunk boundary,

$$\bar{h}_{t+H} = \bar{F}(o_{t+H}, g, p_{t+H}, a_{t+H-k:t+H-1}), \tag{7}$$

with $\bar{F}$ denoting a stop-gradient encoder or a slowly updated target encoder. This avoids the cost of high-frequency video generation while preserving supervision about where the system should be after executing the chunk.

With separate prediction and execution horizons, the transition objective can supervise both the state at the next real replanning point and the longer future structure:

$$\mathcal{L}_{\text{state}} = \lambda_{\text{exec}} \|\hat{h}_{t+H_e} - \text{sg}(\bar{h}_{t+H_e})\|_2^2 + \lambda_{\text{pred}} \|\hat{h}_{t+H_p} - \text{sg}(\bar{h}_{t+H_p})\|_2^2. \tag{8}$$

The H_e term is the closed-loop term: it corresponds to the state at which the robot will re-observe and replan. The H_p term is an auxiliary long-range target that encourages the latent action to preserve future structure beyond the immediately executed segment.

4 Learning Latent Action Codes

4.1 Posterior Latent-Action Encoder

During training, the demonstrated future action chunk is known. A posterior latent-action encoder can therefore infer a compact behavior representation from the current context and the demonstrated chunk:

$$e_t = E_\phi(h_t, a_{t:t+H-1}). \tag{9}$$

The embedding e_t is then converted into a latent action z_t^a, either by vector quantization in the discrete variant or by sampling from a conditional latent distribution in the continuous variant. Given z_t^a, the action-chunk decoder reconstructs the demonstrated chunk:

$$\hat{a}_{t:t+H-1} = D_\theta(h_t, z_t^a). \tag{10}$$

For a deterministic decoder, the reconstruction loss is

$$\mathcal{L}_{\text{chunk}} = \|a_{t:t+H-1} - \hat{a}_{t:t+H-1}\|_2^2. \tag{11}$$

For a diffusion-style decoder, D_θ is replaced by an action denoiser conditioned on (h_t, z_t^a), and the training loss is the standard denoising prediction objective.

4.2 Discrete or Continuous Latent Action Codes

The latent action z_t^a can be represented either as a discrete codebook entry or as a continuous latent vector. In the discrete variant, the posterior encoder first maps the demonstrated chunk to the continuous embedding in Eq. 9. Given a learnable codebook $C = \{c_1, \ldots, c_K\}$, the embedding is quantized to its nearest code,

$$k^\star = \arg\min_k \|e_t - c_k\|_2^2, \qquad z_t^a = c_{k^\star}. \tag{12}$$

The resulting code is used by the action-chunk decoder in Eq. 10. The VQ objective is

$$\mathcal{L}_{\text{VQ}} = \mathcal{L}_{\text{chunk}} + \|\,\text{sg}(e_t) - z_t^a\|_2^2 + \beta_{\text{com}} \|e_t - \text{sg}(z_t^a)\|_2^2, \tag{13}$$

where $\text{sg}(\cdot)$ denotes stop-gradient. The first additional term updates the codebook, while the second is a commitment loss that encourages the posterior embedding to stay close to its selected code. We use a straight-through estimator for the quantized code so that decoder gradients are copied to the posterior embedding e_t.

Because the posterior encoder uses the future action chunk, it is not available at deployment. We therefore train a context-only latent-action prior

$$p_\zeta(k|h_t) \tag{14}$$

over codebook entries. The discrete prior is trained to predict the posterior code assignment,

$$\mathcal{L}_{\text{code-prior}} = CE(k^\star, p_\zeta(\cdot|h_t)), \tag{15}$$

where $k^\star$ is treated as a stop-gradient target. At inference time, the model samples or selects a latent action code from $p_\zeta(k|h_t)$ and decodes an action chunk without observing $a_{t:t+H-1}$. We monitor codebook perplexity and optionally use a code-usage regularizer to prevent collapse to a small number of codes.

A continuous variant can instead use a conditional variational encoder,

$$q_\phi(z_t^a|h_t, a_{t:t+H-1}) = \mathcal{N}(\mu_\phi, \Sigma_\phi), \tag{16}$$

with a learned context-only prior $p_\zeta(z_t^a|h_t)$. The prior is trained with

$$\mathcal{L}_{\text{latent-prior}} = \text{KL}\left(q_\phi(z_t^a|h_t, a_{t:t+H-1}) \,\|\, p_\zeta(z_t^a|h_t)\right). \tag{17}$$

The discrete version is easier to visualize and analyze as an action-token vocabulary, while the continuous version can provide smoother interpolation for continuous control.

4.3 Chunk-Level Future State Prediction

The framework does not require high-frequency visual rollout generation. Instead of predicting every intermediate image frame $\hat{o}_{t+1:t+H}$, we predict only the chunk-boundary control state. A lightweight transition head predicts this boundary state from the current state and latent action:

$$\hat{h}_{t+H} = T_\psi(h_t, z_t^a). \tag{18}$$

When a single chunk horizon is used, the state prediction loss is

$$\mathcal{L}_{\text{state}} = \|T_\psi(h_t, z_t^a) - \text{sg}(\bar{h}_{t+H})\|_2^2. \tag{19}$$

In the receding-horizon variant, Eq. 8 replaces this single-boundary loss. A contrastive variant can replace either squared term by pulling the predicted latent state toward the true boundary state and pushing it away from negative boundary states from other chunks. This objective makes the latent action useful not only for reconstructing the demonstrated action chunk, but also for predicting where the system will be at the next closed-loop replanning point and at the longer prediction boundary. For a pure imitation variant, the transition head is optional; for the full utility-calibrated LARA model, we include it by default because it provides the main chunk-level predictive substrate for scoring candidate experts and routes.

5 Binding Latent Action Codes to Residual Experts

Let f_{sh} be the dense or latent-action policy obtained before expert training. It predicts a shared action chunk

$$a_{\text{sh},t} = f_{\text{sh}}(h_t, z_t^a), \tag{20}$$

which carries the common visuomotor structure needed by most demonstrations. Rather than asking each expert to predict the full chunk from scratch, LARA uses M residual experts $\{\Delta E_m\}_{m=1}^M$. In practice we apply a training schedule and a norm clamp to the residual before adding it to the shared prediction:

$$\tilde{\Delta}E_m(u_t) = s(t)\, \min\left(1, \frac{\Delta_{\max}}{\|\Delta E_m(u_t)\|_2 + \epsilon}\right) \Delta E_m(u_t), \qquad u_t = [h_t, z_t^a], \tag{21}$$

where $s(t) \in [0, 1]$ is a residual warm-up schedule and $\Delta_{\max}$ is the maximum correction norm. The expert-corrected chunk is then

$$\hat{a}_{t:t+H-1}^{(m)} = a_{\text{sh},t:t+H-1} + \tilde{\Delta}E_m(h_t, z_t^a). \tag{22}$$

This parameterization makes overlap explicit: common behavior remains in the shared policy, while routed experts are rewarded for corrections that improve the shared prediction on particular behavior regions.

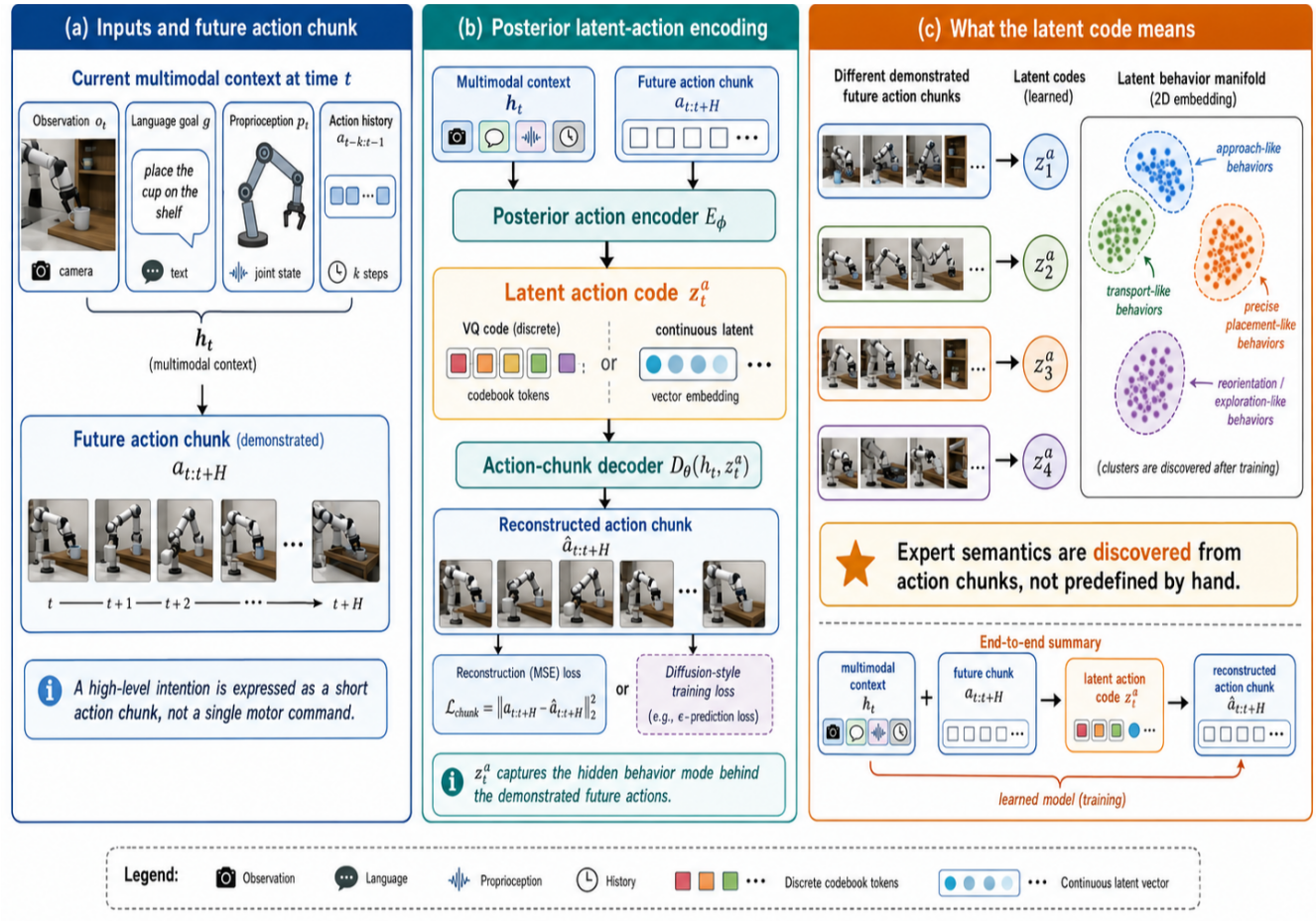

Figure 2: Learning latent action codes. A posterior action encoder observes the current multimodal context and a demonstrated future action chunk, then compresses the chunk into a latent action code. The code may be discrete, as in a VQ codebook, or continuous. It is trained to reconstruct the action chunk and to organize demonstrations into a latent behavior manifold.

For a demonstrated chunk, define the shared loss and expert-corrected loss as

$$\ell_{\text{sh}} = \ell(a_{t:t+H-1}, a_{\text{sh},t:t+H-1}), \qquad \ell_m = \ell(a_{t:t+H-1}, \hat{a}^{(m)}_{t:t+H-1}). \tag{23}$$

The training-time posterior responsibility is assigned by improvement over the shared policy, optionally penalizing large residual corrections:

$$q(m|h_t, a_{t:t+H-1}) = \frac{\exp\left((\ell_{\text{sh}} - \ell_m - \beta\|\tilde{\Delta}E_m(h_t, z_t^a)\|^2)/\tau_q\right)}{\sum_{j=1}^{M} \exp\left((\ell_{\text{sh}} - \ell_j - \beta\|\tilde{\Delta}E_j(h_t, z_t^a)\|^2)/\tau_q\right)}. \tag{24}$$

When no expert improves the shared policy by a useful margin, the route should remain close to the shared action rather than inventing a spurious expert assignment. In implementation this is monitored by expert improvement and residual-norm diagnostics, and can be enforced with a shared-only option or a margin gate.

The residual expert loss is responsibility-weighted:

$$\mathcal{L}_{\text{expert}} = \sum_{i \in \mathcal{B}} \sum_{m=1}^{M} q(m|h_i, a_{i:i+H-1})\, \ell_m(a_{i:i+H-1}; h_i) + \lambda_\Delta \|\tilde{\Delta}E_m(h_i, z_i^a)\|^2. \tag{25}$$

This is a soft assignment, not a hard split of the dataset. A demonstration can update multiple experts, but it updates most strongly the experts that improve the shared latent-action policy. The mechanism resembles soft EM, with the important distinction that responsibilities measure *relative improvement* rather than generic full-action likelihood. This avoids the failure mode where all experts relearn the same dense action head.

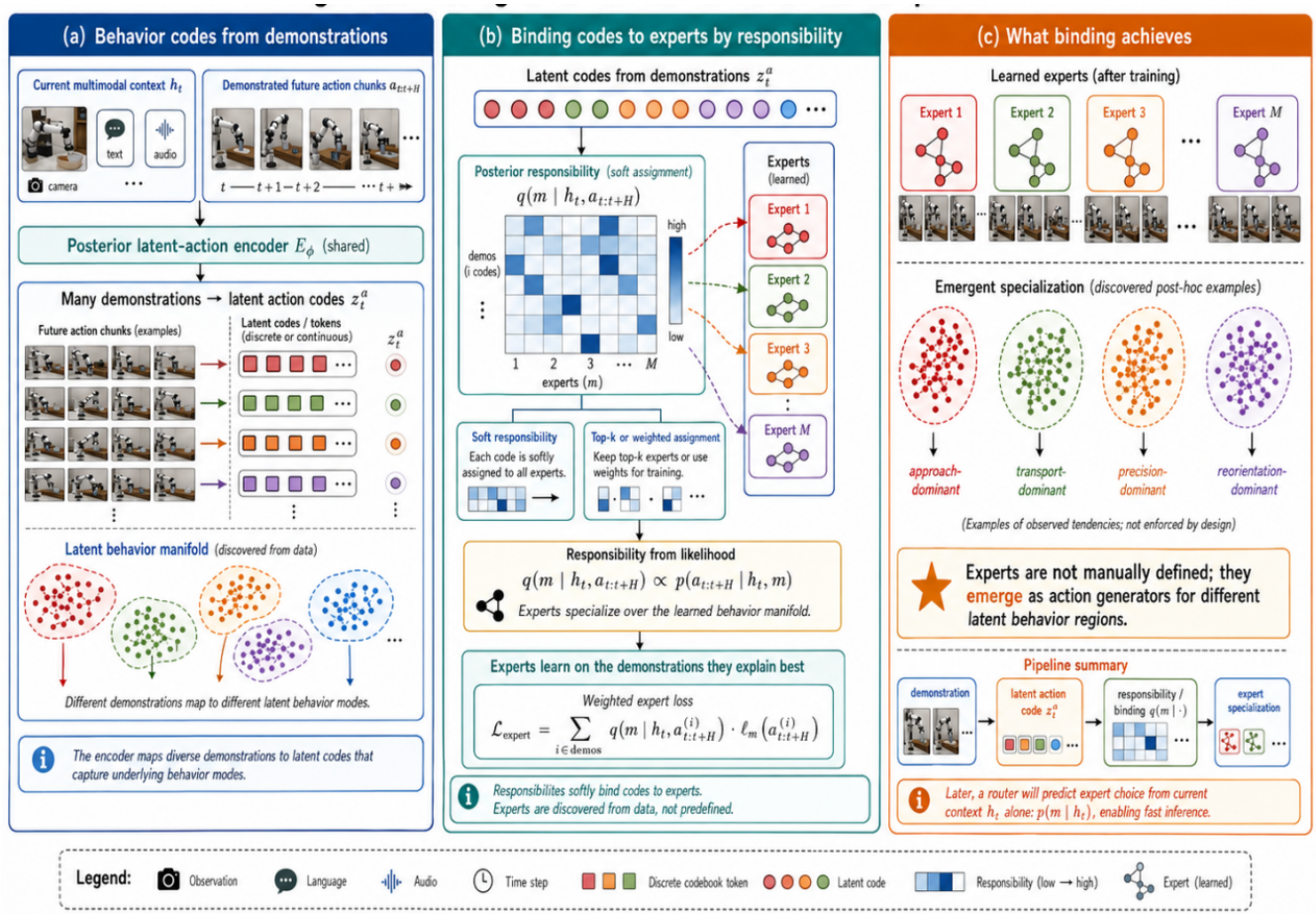

Figure 3: Binding latent action codes to MoE experts. Demonstrations are encoded into latent action codes, then softly assigned to learned experts by posterior responsibility. Experts are not manually named or predefined; they emerge as action generators for different regions of the latent behavior manifold.

This design avoids a hand-coded taxonomy. We do not say that Expert 1 is contact, Expert 2 is transport, or Expert 3 is recovery. If such tendencies appear after training, they are post-hoc interpretations of learned residual behavior regions, not imposed labels.

6 Training the Router

6.1 Posterior Teacher

The responsibility distribution in Eq. 24 is a training-time posterior signal because it depends on the demonstrated future action chunk. We denote it

$$q_{\text{post}}(m|h_t, a_{t:t+H-1}). \tag{26}$$

It answers: *which expert explains the demonstrated future behavior?*

6.2 Episode-Level Pool Router

For memory-constrained deployment, the route predictor can be factorized into two levels. At the beginning of an episode or task, an episode-level pool router chooses a resident expert pool

$$P_\tau = \text{TopD}(p_\chi(m|g, h_1, b), d_\tau), \tag{27}$$

where h_1 is the initial context, g is the goal instruction, and b is an optional deployment budget. The pool size d_τ can be fixed for a target device or sampled during training to support multiple resident-expert budgets. This makes

routing not only sparse per chunk, but also localized over the episode: only experts in P_τ must be resident for that rollout.

The episode-level router is trained to cover all experts that may be needed during the episode, not only the expert that explains the current chunk. A mean responsibility target alone can miss low-frequency but success-critical skills, such as the final insertion or recovery segment after a long approach. We therefore use a coverage-aware episode score

$$s_\tau(m) = \lambda_{\text{avg}} \frac{1}{|\mathcal{T}_\tau|} \sum_{t \in \mathcal{T}_\tau} q_{\text{post}}(m|h_t, a_{t:t+H-1}) + \lambda_{\max} \max_{t \in \mathcal{T}_\tau} q_{\text{post}}(m|h_t, a_{t:t+H-1}) + \lambda_{\text{util}} u_\tau(m), \tag{28}$$

where the average term captures common experts, the max term preserves rare high-responsibility experts, and $u_\tau(m)$ is an optional episode-level utility score from counterfactual or evaluator labels. The normalized pool teacher is

$$q^\tau_{\text{pool}}(m) = \frac{s_\tau(m)}{\sum_j s_\tau(j)}. \tag{29}$$

The pool router is trained by episode-level distillation

$$\mathcal{L}_{\text{pool}} = \text{KL}\left(\text{sg}(q^\tau_{\text{pool}}) \| p_\chi(\cdot|g, h_1, b)\right). \tag{30}$$

We also track resident coverage of the teacher distribution,

$$\text{coverage}(P_\tau) = \sum_{m \in P_\tau} q^\tau_{\text{pool}}(m), \tag{31}$$

and use a differentiable soft coverage objective during training,

$$\mathcal{L}_{\text{cov}} = -\log\left(\sum_m p_\chi(m|g, h_1, b) q^\tau_{\text{pool}}(m) + \epsilon\right). \tag{32}$$

The chunk-level router is then trained on the current chunk posterior within the resident pool. Thus the two levels have distinct labels: the pool router learns episode expert coverage, while the chunk router learns current-chunk expert selection under the selected pool. This pooling mechanism is an implementation choice, not a substitute for latent-action discovery. It should be ablated by reporting closed-loop success as a function of resident expert percentage. Similar subset-retention evaluation is used in modular MoE language modeling, where expert subsets are assessed directly under memory budgets [10].

6.3 Chunk-Level Prior Router

At deployment, the future action chunk is unavailable. The chunk-level router must use only current context and, when enabled, must route within the episode pool. Define masked router logits

$$\tilde{r}^m_\rho(h_t, P_\tau) = \begin{cases} r^m_\rho(h_t), & m \in P_\tau, \\ -\infty, & m \notin P_\tau. \end{cases} \tag{33}$$

The masked route distribution is

$$p_\rho(m|h_t, P_\tau) = \text{softmax}(\tilde{r}_\rho(h_t, P_\tau))_m. \tag{34}$$

Without an episode pool, P_τ is the full expert set. The active route is a sparse top-k set

$$S_t = \text{TopK}(p_\rho(m|h_t, P_\tau), k), \quad S_t \subseteq P_\tau. \tag{35}$$

with normalized weights over selected experts. The action generator is

$$\hat{a}_{t:t+H-1} = \sum_{m \in S_t} \omega^m_t E_m(h_t), \quad \omega^m_t = \frac{\exp(r^m_\rho(h_t)/\tau_r)}{\sum_{j \in S_t} \exp(r^j_\rho(h_t)/\tau_r)}. \tag{36}$$

The chunk router is first trained by distilling the posterior teacher:

$$\mathcal{L}_{\text{distill}} = \text{KL}\left(\text{sg}(q_{\text{post}}(\cdot|h_t, a_{t:t+H-1})) \| p_\rho(\cdot|h_t, P_\tau)\right). \tag{37}$$

Load balancing and route stickiness regularize this phase:

$$\mathcal{L}_{\text{stick}} = \|p_\rho(\cdot|h_t, P_\tau) - p_\rho(\cdot|h_{t-1}, P_\tau)\|_1. \tag{38}$$

Load balancing should be measured across a sufficiently diverse batch of episodes, not forced within a single episode where pool consistency is desired. Stickiness prevents high-frequency route thrashing.

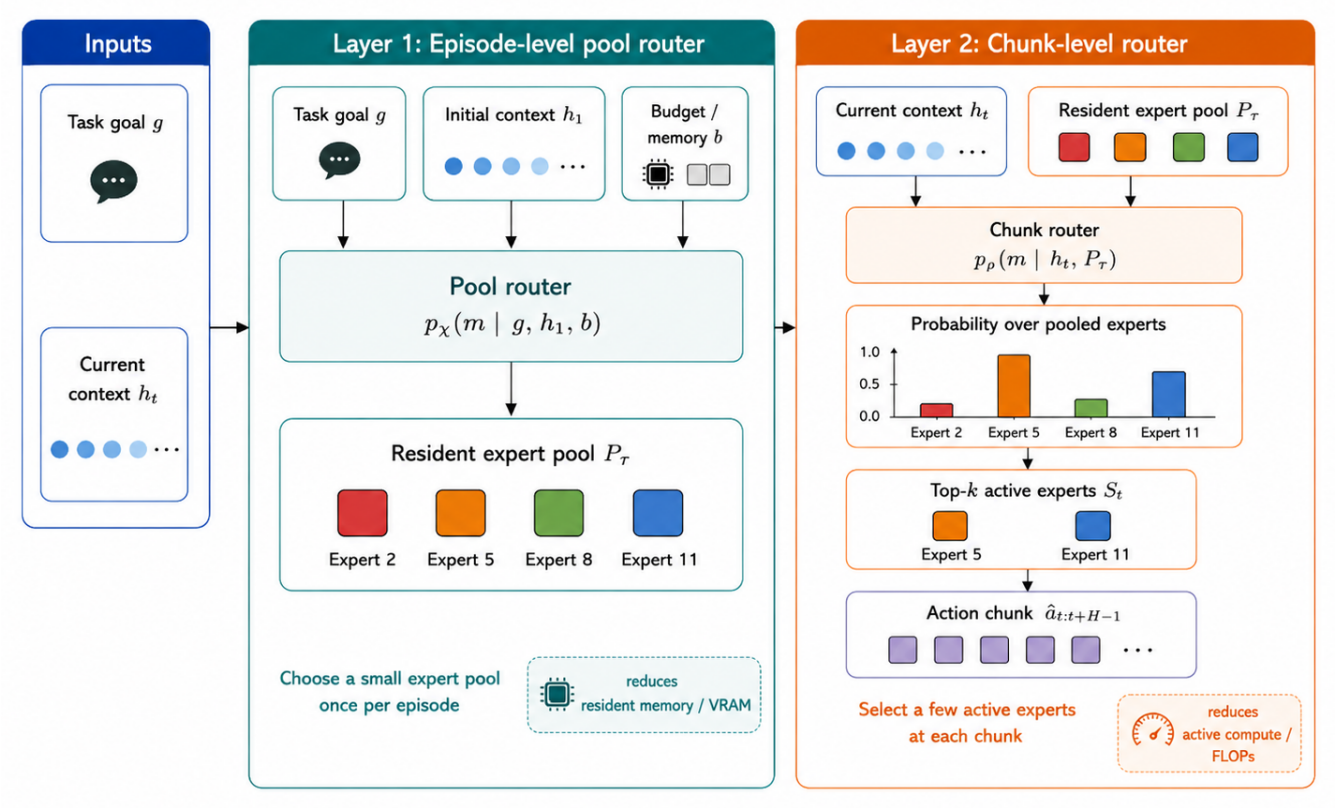

Figure 4: Two-level router for deployment-oriented sparse action generation. The episode-level pool router selects a small resident expert pool P_τ once per task from the goal, initial context, and device budget, reducing resident memory. The chunk-level router then selects a top-k active set $S_t \subseteq P_\tau$ from the current context at each control step, reducing active compute while preserving local flexibility.

6.4 Counterfactual Utility Fine-Tuning

Reconstructing demonstrations is not the same as improving control. The final router objective therefore evaluates candidate experts or routes by a downstream control utility. For a candidate route S at context h_t, define

$$\begin{aligned} J_t(S) = &- \alpha\, \ell_{\text{act}}(\hat{a}^S_{t:t+H-1}, a_{t:t+H-1}) - \alpha_s\, d(\hat{h}^S_{t+H}, \bar{h}_{t+H}) \\ &+ \beta\, V^-(\hat{h}^S_{t+H}, g) - \delta\, \sigma^-(\hat{h}^S_{t+H}, g) - \eta\, \text{Cost}(S). \end{aligned} \tag{39}$$

Here V^- is a lagged value or progress predictor, σ^- is an uncertainty or instability estimate, and $\text{Cost}(S)$ is active inference cost. If sparse success or reward labels are available, V^- may be trained as a success/value head; otherwise it can be replaced by proxy progress, latent-state consistency, or ranking signals. The key difference from video-world-model training is that utility is computed over action chunks and chunk-boundary latent states, not dense visual frame predictions.

For a candidate expert m or route replacement $S_t(m)$, the centered utility target is

$$U^m_t = J_t(S_t(m)) - \frac{1}{|\mathcal{C}_t|} \sum_{j \in \mathcal{C}_t} J_t(S_t(j)), \tag{40}$$

where $\mathcal{C}_t$ is a small candidate set consisting of the current route and sampled alternatives. The utility regression loss is

$$\mathcal{L}_{\text{utility}} = \frac{1}{|\mathcal{C}_t|} \sum_{m \in \mathcal{C}_t} \text{Huber}(r^m_\rho(h_t), \text{sg}(U^m_t)). \tag{41}$$

A pairwise ranking loss can be added to improve top-k ordering:

$$\mathcal{L}_{\text{rank}} = \mathbb{E}_{i,j} \log\left(1 + \exp[-\,\text{sg}(U^i_t - U^j_t)(r^i_\rho(h_t) - r^j_\rho(h_t))]\right). \tag{42}$$

6.5 Collapse Prevention and Routing Stabilization

Sparse latent-action routing can fail by collapsing to a small set of codes, experts, or routes. We therefore stabilize training at several levels rather than relying on load balancing alone.

Latent-code usage. For the discrete variant, we monitor the code usage distribution $\bar{p}_{\text{code}}(k)$ and its perplexity,

$$\text{PPL}_{\text{code}} = \exp\left(-\sum_{k=1}^{K} \bar{p}_{\text{code}}(k) \log \bar{p}_{\text{code}}(k)\right). \tag{43}$$

Low perplexity or a high dead-code ratio indicates codebook collapse. We optionally add a weak code-usage regularizer

$$\mathcal{L}_{\text{code-bal}} = \text{KL}\left(\mathbf{u}_K \,\|\, \bar{p}_{\text{code}}\right), \tag{44}$$

where $\mathbf{u}_K$ is the uniform distribution over codebook entries. In implementation, EMA codebook updates or dead-code reinitialization can be used when many codes remain unused for long intervals.

Responsibility smoothing. Expert specialization is driven by q_{post}. If the responsibility distribution becomes too sharp early in training, a slightly better expert can absorb most chunks before other experts learn. If it remains too diffuse, experts can all learn the same correction. We therefore treat temperature and top-r assignment as bootstrapping tools rather than evidence of specialization: early training may restrict responsibility to several high-improvement experts, and later stages relax this constraint to test whether the learned residual preferences persist. We also allow a small uniform floor,

$$\tilde{q}_{\text{post}}(m|h_t, a_{t:t+H-1}) = (1-\epsilon) q_{\text{post}}(m|h_t, a_{t:t+H-1}) + \epsilon/M, \tag{45}$$

or a top-r soft assignment that updates several high-responsibility experts instead of using a hard top-1 assignment. These choices give non-dominant experts gradient signal during early training, but they do not by themselves prove complementary experts.

Expert and router balance. For chunk-level routing, balancing should be measured across a diverse batch of episodes rather than inside a single episode. For episode-level pools, balancing is likewise computed over many trajectories. A generic form is

$$\mathcal{L}_{\text{lb}} = \text{KL}(\mathbf{u}_M \,\|\, \bar{p}_{\text{route}}) + \text{KL}(\mathbf{u}_M \,\|\, \bar{p}_{\text{pool}}), \tag{46}$$

where $\bar{p}_{\text{route}}$ is the average chunk-level route distribution and $\bar{p}_{\text{pool}}$ is the average episode-level pool distribution. Early training can also use router entropy regularization or noisy top-k routing to encourage exploration. Route stickiness is kept as a separate stabilizer because it prevents high-frequency route thrashing without forcing all experts to be used inside one trajectory.

Expert diversity and capacity. Balanced usage does not guarantee distinct expert functions. In the residual formulation, diversity should be measured on corrections rather than final actions, because many states legitimately require similar total motor commands. We monitor residual norm, per-expert improvement over the shared policy, and pairwise similarity of residual outputs on shared inputs. We optionally add a weak decorrelation penalty,

$$\mathcal{L}_{\text{div}} = \frac{1}{M(M-1)} \sum_{m \neq n} \left(\frac{\Delta E_m(u_t)^\top \Delta E_n(u_t)}{\|\Delta E_m(u_t)\| \, \|\Delta E_n(u_t)\|} \right)^2, \quad u_t = [h_t, z_t^a]. \tag{47}$$

Expert dropout or soft capacity limits can further prevent early dominant experts from consuming most high-responsibility chunks. A healthy residual expert set should show nonzero residual norms, positive improvement for at least some contexts, and residual similarity below the fully redundant regime.

Residual magnitude control. Residual experts can also fail in the opposite direction: once they receive enough posterior mass, their corrections can grow until they overwrite the shared latent-action policy instead of improving it. We therefore treat residual magnitude as a first-class stability constraint. During expert warm-up, the effective residual scale $s(t)$ increases gradually from zero to its target value, and the applied correction is clipped by the norm constraint in Eq. (21). The raw residual is still logged, but the policy only receives the clamped residual. We monitor the applied residual norm, raw residual norm, clamp rate, expert active rate, shared-only rate, effective

number of posterior experts, and mean improvement $\ell_{\text{sh}} - \ell_m$. A useful residual expert should keep nonzero expert participation and non-collapsed top-r assignment while maintaining nonnegative or near-zero improvement over the shared policy. Persistent negative improvement together with rising raw residual norm indicates over-correction, even if routing entropy or load balance looks healthy.

Delayed utility calibration. Counterfactual utility should not dominate before the latent codes, experts, and prior router are reasonably stable. We therefore start with latent-action reconstruction and posterior-responsibility training, then add prior-router distillation, and only later increase the weight of $\mathcal{L}_{\text{utility}}$. Utility targets use stop-gradient labels and lagged value or uncertainty heads. We retain a nonzero distillation weight during utility fine-tuning so that routing remains close to the demonstrated behavior distribution while being calibrated toward downstream control.

We collect optional stabilization terms as

$$\mathcal{L}_{\text{stab}} = \lambda_{\text{code}}\mathcal{L}_{\text{code-bal}} + \lambda_{\text{div}}\mathcal{L}_{\text{div}} + \lambda_{\text{ent}}\mathcal{L}_{\text{ent}}, \tag{48}$$

where $\mathcal{L}_{\text{ent}}$ denotes an early-stage router-entropy term. These terms are treated as training stabilizers and should be ablated separately from the core latent-action and utility-routing claims.

7 Full Objective and Algorithm

The full training objective is

$$\begin{aligned}\mathcal{L} =& \mathcal{L}_{\text{chunk}} + \lambda_z\mathcal{L}_{\text{latent}} + \lambda_s\mathcal{L}_{\text{state}} + \lambda_e\mathcal{L}_{\text{expert}} + \lambda_p\mathcal{L}_{\text{pool}} + \lambda_d\mathcal{L}_{\text{distill}} \\ &+ \lambda_u\mathcal{L}_{\text{utility}} + \lambda_r\mathcal{L}_{\text{rank}} + \lambda_{\text{lb}}\mathcal{L}_{\text{lb}} + \lambda_{\text{st}}\mathcal{L}_{\text{stick}} + \lambda_{\text{stab}}\mathcal{L}_{\text{stab}}.\end{aligned} \tag{49}$$

The latent regularizer $\mathcal{L}_{\text{latent}}$ contains the codebook, commitment, and code-prior terms for the discrete variant, or the variational prior KL for the continuous variant. The reconstruction term $\mathcal{L}_{\text{chunk}}$ is written separately to avoid double counting. If the episode pool is not used, $\lambda_p = 0$ and P_τ is the full expert set.

1. Sample demonstration chunks $(o_t, g, p_t, a_{t-k:t-1}, a_{t:t+H_p-1}, o_{t+H_e}, p_{t+H_e}, o_{t+H_p}, p_{t+H_p})$.
2. Encode current and boundary states: $h_t = F_\theta(o_t, g, p_t, a_{t-k:t-1})$, $\bar{h}_{t+H_e} = \bar{F}(o_{t+H_e}, g, p_{t+H_e}, a_{t+H_e-k:t+H_e-1})$, and $\bar{h}_{t+H_p} = \bar{F}(o_{t+H_p}, g, p_{t+H_p}, a_{t+H_p-k:t+H_p-1})$.
3. Infer posterior latent action embedding $e_t = E_\phi(h_t, a_{t:t+H_p-1})$ and obtain z_t^a by quantization or continuous sampling.
4. Train action decoder and chunk-boundary transition with $\mathcal{L}_{\text{chunk}}$ and $\mathcal{L}_{\text{state}}$.
5. Freeze or slow-update the shared latent-action policy and compute $a_{\text{sh}} = f_{\text{sh}}(h_t, z_t^a)$.
6. Predict raw residual corrections $\Delta E_m(h_t, z_t^a)$, apply residual warm-up and norm clipping to obtain $\tilde{\Delta}E_m(h_t, z_t^a)$, and form expert-corrected chunks $\hat{a}^{(m)} = a_{\text{sh}} + \tilde{\Delta}E_m(h_t, z_t^a)$.
7. Compute posterior expert responsibilities from improvement over the shared loss, $q_{\text{post}}(m|h_t, a_{t:t+H_p-1})$.
8. Update residual experts with responsibility-weighted loss $\mathcal{L}_{\text{expert}}$ and raw/applied residual norm, clamp-rate, improvement, and diversity diagnostics.
9. Optionally aggregate responsibilities over the episode and train $p_\chi(P_\tau|g, h_1, b)$ with $\mathcal{L}_{\text{pool}}$.
10. Train the chunk-level prior router $p_\rho(m|h_t, P_\tau)$ by distillation from q_{post}.
11. Apply stabilization terms, including code-usage monitoring, smoothed responsibilities, global load balancing, route stickiness, and optional expert-diversity regularization.
12. After warm-up, evaluate a small set of candidate routes using $J_t(S)$ and train router scores with $\mathcal{L}_{\text{utility}}$ and $\mathcal{L}_{\text{rank}}$.

Algorithm 1: Training Latent-Action Routing

At inference time, the system executes only the context encoder, deployable router, selected experts, and action decoder:

$$(g, h_1, b) \to P_\tau, \qquad h_t \to S_t \subseteq P_\tau \to \hat{a}_{t:t+H_p-1}. \tag{50}$$

Only the prefix $\hat{a}_{t:t+H_e-1}$ is executed. The next policy call starts from the real observation and proprioception at $t + H_e$, not from a predicted latent state:

$$o_{t+H_e}, p_{t+H_e} \rightarrow h_{t+H_e} \rightarrow \hat{a}_{t+H_e:t+H_e+H_p-1}. \tag{51}$$

If the episode pool is disabled, P_τ is the full expert set. The posterior action encoder, future-action teacher, and counterfactual candidate scoring are training-time mechanisms. Predicted latent states are used for training and utility calibration, not as the next round's real closed-loop input.

Current Repository Implementation Status

The current repository has completed the SO101 VLA-JEPA action path with LARA components enabled by default. It reuses the pretrained VLA representation, extracts latent and embodied action tokens from the Qwen/V-JEPA token stream, and routes those tokens through the latent-action head, transition head, MoE/router, residual or direct action experts, and utility-proxy losses before predicting continuous follower-arm action chunks. The SO101 dataloader and action adapter also propagate future-action validity masks so trajectory-end padding does not supervise the action loss or any latent, transition, utility-proxy, or direct-expert losses. The training code now separates prediction-horizon and execution-horizon evaluation metrics on an independent validation dataloader when `datasets.vla_eval_data` is configured, and it records the world-model loss with a configurable `trainer.loss_scale.wm` weight rather than a hard-coded multiplier.

The latent-action, MoE, and two-level routing components described in the preceding sections are not yet completed as a validated implementation of LARA. The codebase contains experimental scaffolding for a posterior latent action head with an explicit action-chunk decoder and reconstruction loss, residual action experts over a shared latent-action baseline, direct full-action experts for ablation, posterior-responsibility signals, an episode-level resident-pool router, a chunk-level top-k router inside that pool, route diagnostics, and counterfactual-utility sidecar plumbing. These paths are now default-on in the LIBERO100 experiment configuration and should be treated as active research code rather than completed paper evidence. In particular, the current repository does not yet provide a trained and validated MoE action head, a validated two-level routing policy, real counterfactual utility labels, or closed-loop SO101 results demonstrating resident-pool reuse and sparse expert selection under matched compute.

The implementation exposes four explicit SO101 stage configurations: `lara_so101_baseline.yaml`, `lara_so101_latent_vq.yam` `lara_so101_moe_direct.yaml`, and `lara_so101_utility_pool.yaml`. The historical baseline filename is kept for compatibility, but these configs now enable the latent-action, direct-MoE, and utility/pool paths by default. They do not by themselves constitute completed LARA evidence.

Protocol tooling can summarize rollout JSON/JSONL records into matched-budget rows, subset-retention curves, route-sequence diagnostics, and compute-success Pareto flags, but it does not itself produce real FLOP, latency, memory, or closed-loop success measurements. The deployment websocket server can optionally write raw route traces and server-side latency/VRAM measurements to JSONL, and a strict evidence-audit mode can reject incomplete rollout records. These tools are intended to prevent protocol summaries from being mistaken for completed LARA evidence. Full latent-action training, MoE expert validation, posterior-to-router distillation, episode-pool routing, chunk-level routing, and counterfactual utility calibration remain future implementation and evaluation work.

8 Experimental Protocol

8.1 Data and Language Assumption

The main protocol should use ordinary goal-level language and trajectories. No dense step-level language labels are required. Each episode contains

$$(g, o_{1:T}, p_{1:T}, a_{1:T}). \tag{52}$$

Optional signals such as sparse reward r_t, episode success y_τ, or termination annotations δ_t may be recorded by a benchmark, simulator, or external evaluator, but the primary representation-learning pipeline does not require them. Trajectory boundaries are handled as data boundaries: chunks that extend beyond the recorded trajectory are masked out of the reconstruction and transition losses. A recommended ablation compares: (i) goal-only language, (ii) goal plus terminal success condition when available, and (iii) dense step annotations. The primary method should not depend on (iii), and should report whether utility heads use true success labels or proxy progress signals.

Table 1: Primary matched-compute results. Entries are placeholders to be filled by completed runs.

Benchmark	Method	Success ↑	Return ↑	Resident Params ↓	FLOPs ↓	Latency ↓	Route Regret ↓
LIBERO	Dense VLA	TBD	TBD	TBD	TBD	TBD	–
LIBERO	Generic MoE	TBD	TBD	TBD	TBD	TBD	TBD
LIBERO	Latent-Action MoE	TBD	TBD	TBD	TBD	TBD	TBD
LIBERO	LARA	TBD	TBD	TBD	TBD	TBD	TBD
LIBERO	LARA + route pool	TBD	TBD	TBD	TBD	TBD	TBD
CALVIN	Dense VLA	TBD	TBD	TBD	TBD	TBD	–
CALVIN	Generic MoE	TBD	TBD	TBD	TBD	TBD	TBD
CALVIN	Latent-Action MoE	TBD	TBD	TBD	TBD	TBD	TBD
CALVIN	LARA	TBD	TBD	TBD	TBD	TBD	TBD
CALVIN	LARA + route pool	TBD	TBD	TBD	TBD	TBD	TBD

8.2 Benchmarks

The minimal experimental stack is:

- **ManiSkill3**: fast sanity checks and ablations over M, k, chunk horizon H, utility components, and route-pool budgets.
- **LIBERO**: language-conditioned manipulation and transfer splits.
- **CALVIN**: long-horizon multi-step manipulation.

A real-data ranking track, such as DROID, BridgeData V2, robomimic, or Open X-Embodiment subsets, can be added for policy-ranking and route-calibration analysis if compute allows.

8.3 Baselines

The primary causal baselines are:

1. Dense VLA with the same backbone and action-chunk decoder.
2. Generic sparse MoE-VLA with load balancing but no latent-action posterior responsibility.
3. Latent-action MoE without utility fine-tuning.
4. LARA without episode-level pool routing.
5. LARA with two-level route-pool deployment.
6. Dense-large with matched total parameters, when feasible.
7. Dense-small with matched active parameters.

All primary comparisons must report total parameters, resident parameters, active parameters, inference FLOPs, latency, and VRAM.

8.4 Metrics

The primary metric is closed-loop success under matched active inference compute. Secondary metrics are return, sample-efficiency AUC, compute-success Pareto efficiency, policy-ranking Spearman ρ and Kendall τ, posterior-router KL, teacher mass@resident-pool, teacher mass@active-top-k, critical expert miss rate, top-k route consistency, route regret, expert usage entropy, residual improvement over the shared policy, residual norm, pairwise residual similarity, route-switch frequency, utility calibration error, train-time overhead, and inference latency. If the episode-level route pool is used, also report subset-retention curves: success versus percentage of resident experts retained, e.g., 100%, 50%, 25%, and 12.5% resident experts.

8.5 Success and Failure Conditions

The main claim should be accepted only if the full method improves closed-loop success over both Dense VLA and Generic MoE under matched active compute, and also improves at least one of policy-ranking fidelity, route calibration, or route regret. If it beats Dense VLA but not Generic MoE, the evidence supports sparsity but not

Table 2: Ablation plan for isolating the causal role of latent action discovery, expert binding, utility-calibrated routing, and deployment-oriented route-pool choices.

Block	Variant	Question	Success ↑	Rank ρ ↑
Representation	No latent action code	Does chunk latent modeling matter?	TBD	TBD
Binding	Hard cluster assignment	Is soft responsibility needed?	TBD	TBD
Binding	Generic MoE only	Is the gain just sparsity?	TBD	TBD
Router	Distillation only	Is posterior-to-prior routing sufficient?	TBD	TBD
Router	Utility only	Does utility work without teacher warm-up?	TBD	TBD
Router	Full distill + utility	Does complete route calibration help?	TBD	TBD
Deployment	No episode pool	Does ordinary chunk routing suffice?	TBD	TBD
Deployment	Fixed pool size	Is a fixed resident budget brittle?	TBD	TBD
Deployment	Randomized pool size	Does budget randomization improve retention curves?	TBD	TBD
Language	Goal-only	Are step labels necessary?	TBD	TBD
Language	Goal + terminal condition	Does outcome text help?	TBD	TBD
Language	Dense step text	Does step supervision overfit routing?	TBD	TBD

latent-action responsibility or utility routing. If the route-pool variant improves resident-parameter efficiency but not control, the result should be framed as a deployment optimization rather than as the core learning claim.

9 Discussion

The revised framework changes the interpretation of experts. They are no longer hand-labeled manipulation phases. They are learned action generators tied to regions of a latent behavior manifold. This better matches the intuition that an embodied agent should reason at the level of goals and short-horizon behavior modes, not explicit per-step muscle-like instructions.

The framework also narrows the role of world modeling. It does not require dense visual prediction. For control, the useful prediction target can be a chunk-boundary latent state, a value or success estimate, and an uncertainty estimate. This makes the method more deployable than high-frequency video generation while preserving enough structure for counterfactual utility evaluation.

The two-level routing implementation connects active-compute efficiency with resident-memory efficiency. Chunk-level top-k routing reduces activated computation, while the episode-level pool can reduce which experts need to be resident during a rollout. These benefits must be measured separately. A method may be FLOP-efficient but not VRAM-efficient if all experts remain resident, so the experimental protocol reports both active parameters and resident parameters.

10 Limitations

The method does not bypass data limitations. If the demonstrations do not cover the necessary behavior modes, latent-action discovery cannot invent them. MoE also does not automatically reduce VRAM: if all experts are resident on the device, memory may increase, even if active FLOPs decrease. Practical deployment should prefer shared VLA backbones with small MoE adapters or action heads. Finally, the posterior teacher can produce misleading responsibilities if the experts are weak early in training; curriculum, entropy regularization, warm-started dense action models, residual warm-up, norm clipping, and explicit collapse diagnostics are likely necessary. Stabilizers such as load balancing or diversity penalties must be tuned carefully, since overly strong regularization can force artificial expert usage and degrade control performance.

11 Conclusion

We proposed LARA, Latent Action Routing for embodied VLA control. The method learns latent action codes from demonstrated action chunks, binds them to experts by posterior responsibility, distills expert choice into a deployable router, and calibrates the router with counterfactual control utility. A deployment-oriented implementation can factor routing into episode-level resident expert selection and chunk-level top-k activation. The resulting system uses ordinary goal-level language and trajectories, avoids hand-designed expert taxonomies, and avoids expensive dense visual rollout generation. Its empirical test is clear: under matched active compute and resident-expert budgets, it should improve closed-loop control and route quality relative to dense VLA and generic sparse MoE baselines.

References

[1] Noam Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*, 2017.

[2] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 2022.

[3] Cheng Chi et al. Diffusion Policy: Visuomotor policy learning via action diffusion. *RSS*, 2023.

[4] Anthony Brohan et al. RT-1: Robotics Transformer for real-world control at scale. arXiv:2212.06817, 2022.

[5] Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *CoRL*, 2023.

[6] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. arXiv:2406.09246, 2024.

[7] Octo Model Team et al. Octo: An open-source generalist robot policy. arXiv:2405.12213, 2024.

[8] Danijar Hafner et al. Mastering diverse domains through world models. arXiv:2301.04104, 2023.

[9] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. *ICLR*, 2024.

[10] Ryan Wang, Akshita Bhagia, and Sewon Min. EMO: Pretraining Mixture of Experts for Emergent Modularity. arXiv:2605.06663, 2026.